

阶段性总结报告

2026 年 1 月 13 日

1 引言

1.1 项目背景与主要目标

在智能制造与工业 4.0 背景下，传统自动化立体仓库调度系统依赖静态规则，难以适应多场景、高并发与动态生产计划需求，制约了系统整体效率与稳定性。

本项目的主要目标如下：

- 设计并实现一套面向自动化立体仓库的智能调度算法包，实现入库、出库及混合调度的统一优化决策；
- 基于仓库实时状态与任务信息，提升出入库效率，降低任务完工时间，提高系统整体吞吐能力；
- 实现堆垛机负载均衡，减少设备空闲与拥堵现象，提升系统运行稳定性；
- 支持多种典型立体库业务场景（如纵梁库、车身库）的参数化适配与扩展；
- 通过标准化接口与现有 WCS 系统集成，确保算法具备工程可落地性与长期可维护性。

1.2 前期调研与数据分析

团队成员前期均已参与过线下调研，参观了主要仓库系统，并了解其运行机制。通过对历史运行数据的分析，发现以下关键问题：

- 调度策略静态，缺乏对实时任务状态、设备负载与库存分布的全局感知能力；
- 堆垛机任务分配不均，易出现局部拥堵与设备空闲并存的现象；
- 未充分考虑多场景业务约束，导致配对失败、频繁移库等无效作业；
- 难以适应生产计划动态变化，影响物料准时供应与系统整体稳定性。

1.3 项目主要目标

为验证算法包的有效性与工程可行性，项目设置如下可量化指标作为评估标准：

- 出入库效率提升：在不新增硬件设备的前提下，单位时间内完成的出入库任务数量较现有系统提升不少于 10%；新增硬件后，出库速率需从 40JPH 提升至 50JPH。

- 纵梁配对成功率：针对配对型立体库场景，入库纵梁一次性配对成功率由原有约 70% 提升至不低于 95%；
- 设备负载均衡性：各巷道堆垛机利用率差异控制在 $\pm 10\%$ 以内，避免长期拥堵或闲置；
- 无效等待时间：堆垛机因调度不合理产生的空闲等待时间较原系统减少不少于 20%；
- 系统响应性能：调度接口平均响应时间控制在 2 秒以内，接口调用成功率不低于 99.3%；
- 系统稳定性：算法集成后系统可长期稳定运行，不因调度逻辑导致严重故障或业务中断。

现阶段团队主要进行的是带有配对约束的纵梁立体库分配和调度策略的开发，并且已经基本实现了对应的仿真环境搭建以及一些策略间的可视化对比。

2 仿真环境与流程（事件驱动）

2.1 仓库与设备参数

仿真环境包含仓库结构、设备与时间参数等配置项：

- 仓库结构：巷道数、行/列/层数、总货位数、最大梁数。
- 货位模式：支持双层货位，上下层可放置不同 SKU；同时添加对单层货位设置的支持。
- 设备与时间参数：磁力吊启用与否、磁力吊工作时间、出库口拥堵时间、入库运输延迟、堵塞时间、移库延迟等。

2.2 生产计划与入库任务来源

- 生产计划：按产线 `production_line` 分组的出库任务序列（JSON）。
- 入库任务：包含到达时间、SKU 列表与入库线 `in_line`（JSON）。
- 初始库存：主要通过 `initial_inventory_count` 进行调整。对于单日的仿真模式，使用“首个出库任务开始之前到达的入库数量”作为初始库存规模，确保与当日任务节奏一致。

2.3 事件定义与触发逻辑

仿真采用事件驱动机制，核心事件包括：

- **EVENT_INBOUND_UNASSIGNED**：入库任务尚未分配巷道。
由 Core 中使用的 Allocator 分配巷道后生成 **EVENT_INBOUND_ARRIVAL_AT_AISLE**。
- **EVENT_INBOUND_ARRIVAL_AT_AISLE**：入库任务到达巷道入口，进入等待队列并触发调度。
- **EVENT_TASK_COMPLETE**：任务完成事件（入库/出库）。入库写入库存，出库扣减库存并触发拥堵/磁力吊时序，完成后触发调度。
- **EVENT_CONGESTION_CLEAR**：出库拥堵解除，允许后续出库派发。

2.4 调度与派发流程

- 调度触发点：每次事件处理后，以及事件队列为空时执行一次“补触发”。
- 空闲巷道派发：
 - 动态生成可派出库任务（满足产线组顺序与库存约束）。
 - 入库任务按“巷道 + 入库线队首”进入候选集。
 - 调度器输出巷道任务序列，派发下一任务并生成 `task_complete` 事件。

2.5 运行模式与侧重点

- `run_compare`（对比实验模式）：
 - 目的：在同一组输入配置下，对比不同策略组合（proposed/baseline 与 heuristic/optimization）的相对表现。
 - 流程特点：一次运行中循环四组配置，产生日志用于横向对比与可视化（如完工时间、配对率、巷道利用率、移库次数）。
 - 侧重点：算法对比、策略敏感性、单次配置的对照实验。
- `run_daily`（单日批量模式）：
 - 目的：评估策略在不同日期数据分布下的稳定性与泛化能力。
 - 流程特点：按日期加载对应的入库/生产计划配置，生成“按天”的日志目录，并在每个日期下执行四种策略组合。
 - 侧重点：跨天鲁棒性、趋势分析、日尺度统计指标对比。

2.6 终止与收尾

- 当日事件队列耗尽且无法触发新任务时结束日内循环。
- 对比实验模式下出库结束后仍有入库残留时，执行一次收尾补处理以完成剩余入库（模拟晚上仅入库的阶段），单日批量模式下由于梁是根据出库任务匹配好的，所有不存在收尾阶段。

2.7 可视化内容（按运行模式区分）

- `run_compare`（`visualize_results.py`）：
 - 完成率对比：统计每个日期下的出库完成率（完成组数/计划组数）。
 - 配对率曲线：按时间轴绘制各种不同计算方式下配对率的动态变化。
 - 每日出库完工时间：每日报告“最后一个出库任务完成时间”。
 - 按天移库次数：从日志中提取每日移库数。
 - 平均巷道利用率（按天）：利用任务持续时间计算巷道占用率并给出平均值。
- `run_daily`（`visualize_daily_results.py`）：

- 完成率对比：统计每个日期下的出库完成率（完成组数/计划组数）。
- 日内结束时间：每个日期的仿真结束时间（小时）。
- 移库次数：每个日期的累计移库次数，并在图例中显示均值。
- 平均巷道利用率：每个日期按日志任务时间段合并后计算占用率，并显示均值。

3 算法改进细节

3.1 当前算法整体思路

- 分配策略（入库侧）：在 `allocate_inbound_aisle` 阶段，根据 SKU-产线映射与产线需求强度选择巷道，并将已排入巷道的入库 SKU 纳入统计，避免同一巷道持续堆积。
- 调度策略（入/出库混排）：在 `decide_for_idle_aisles` 触发时生成候选任务，结合库存可用性、产线组顺序约束与拥堵状态，由调度器输出巷道序列。
- 事件驱动执行：每次事件完成后更新库存与拥堵/磁力吊状态，并再次触发调度，形成“事件—调度—执行”的闭环。

3.2 分配策略对比

- 巷道分配更贴合生产计划：在分配阶段引入 SKU-产线映射与产线当前需求（任务组进度），优先将入库导向更“缺该 SKU”的巷道，减少后续跨巷道搬运。
- 配对优先机制更明确：在候选巷道评估中加入“可配对货位/配对 SKU”统计，使入库更倾向于形成可直接出库的上下层组合。在发现无法完美配对时，加入对“同巷道移库”操作的优先级，也就是说分配策略会更倾向于将无法配对的纵梁放入有其配对纵梁的巷道（处于同巷道另外一排，导致无法配对），便于移库时进行“同巷道移库”操作，减少跨巷道搬运造成的产线堵塞。
- 负载均衡更充分：综合“目标产线负载 + 空位数量 + 已分配入库 SKU”排序，避免单巷道过载导致排队等待。
- 考虑已分配入库任务：在巷道选择时把已经安排给该巷道的入库任务中的 SKU 也计入负载统计，降低连续入库集中到同一巷道的概率。

3.3 调度策略对比

- 全局序列优化：调度器在生成 `aisle_task_sequences` 时对候选序列进行综合评估，而不仅是局部贪心排序。
- 更少的冲突与阻塞：在调度阶段显式考虑拥堵/磁力吊状态（`EVENT_CONGESTION_CLEAR` 释放）与产线组顺序约束，减少“可派但不可执行”的冲突。
- 利用率更均衡：通过全局评估更均匀地分配巷道任务，避免“部分巷道长时间排队、部分巷道空闲”的极端情况。
- 对入库/出库混排更稳健：在评估中同时考虑入库耗时与出库拥堵代价，使混合作业下的完工时间更稳定。

4 初步结果与分析

4.1 仿真设置

- 数据集：。
- 配置文件：。
- 运行模式：。
- 对比实验：。

4.2 结果对比